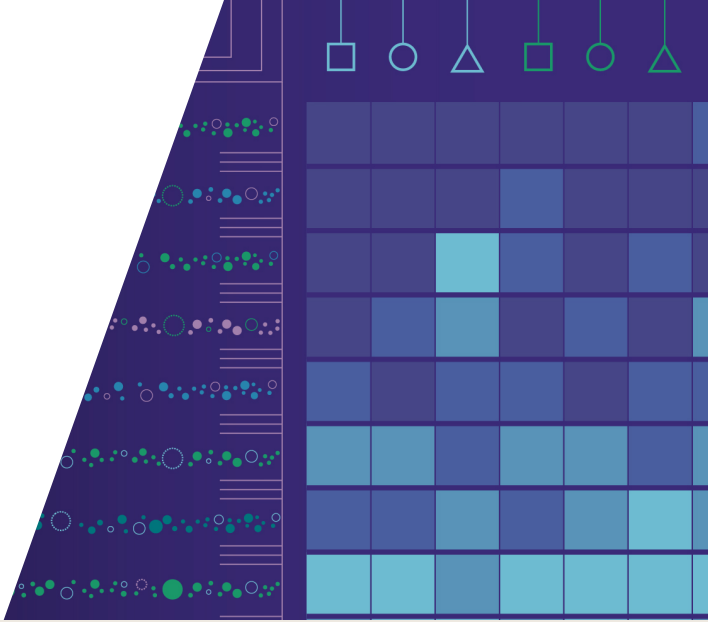

Evaluation and Benchmarking

Chapter Summary

Marcel Wever

Overview: Chapter 2 – Evaluation and Benchmarking

This chapter covered the following topics:

1. **The Big Picture** – Why evaluation is fundamental (and hard)
2. **Evaluation of ML Models** – Estimand, resampling strategies, bias-variance tradeoff
3. **Statistical Tests** – When is a difference real?
4. **Nested Evaluation for AutoML** – The optimistic bias problem and its remedy
5. **Visualizing AutoML Runs** – Anytime curves and uncertainty bands
6. **Benchmarking AutoML** – Rigorous multi-task comparison

1. The Big Picture

Why is evaluation the backbone of ML and AutoML?

- We want models that *generalise* to unseen data – evaluation is our only proxy
- Data is finite, models are stochastic, small design choices change conclusions

Two kinds of overfitting:

Overfitting	A single model fits training data too closely; fails on unseen data
Meta-overfitting	An AutoML system exploits noise in validation scores; the *search process* itself overfits to $\mathcal{D}_{\text{val}}$

- Automation *amplifies* weak evaluation: the more configurations explored, the more extreme the selected noise

2. Evaluation of ML Models

The estimand: expected generalisation risk at a fixed training size n:

$$\mathcal{R}^{(n)}_{\text{gen}}(\boldsymbol{\lambda}) := \mathbb{E}_{\mathcal{D}_{\text{train}} \sim P^n}[\mathcal{R}_{\text{gen}}(\mathcal{I}(\mathcal{D}_{\text{train}}, \boldsymbol{\lambda}))]$$

Three core resampling strategies:

Scheme	Bias	Variance	Cost
Hold-out	Low	High	$1\times$
k-fold CV	Low	Lower	$k\times$
MCCV	Low	Lower	$R\times$

Rules of thumb: Hold-out for very large N; 5–10 fold CV for medium N; MCCV/repeated CV for small N.

Never use test data for any model decision.

3. Statistical Tests

Core question: Does an observed performance difference reflect a real effect, or only random variation?

Key concepts:

- H_0: no difference; α fixed *before* testing (commonly 0.05)
- **Type I error**: false positive (reject H_0 when true) – controlled by α
- **Type II error**: false negative (fail to reject H_0 when false) – reduced by more data

Choosing the right test:

- *Two configurations, paired:* Wilcoxon Signed-Rank test (robust default) or corrected *t*-test
- *More than two:* Friedman omnibus test first, then post-hoc pairwise tests (Holm or Nemenyi) with multiple-testing correction; visualize with critical difference diagrams

Never run pairwise tests without correction, and never fix α after seeing the results.

4. Nested Evaluation for AutoML

The problem: AutoML selects $\boldsymbol{\lambda}^*$ based on $\mathcal{D}_{\text{val}} \Rightarrow$ the selected configuration is data-dependent $\Rightarrow$ naive validation scores are optimistically biased. Bias grows with the number of evaluated configurations T.

The solution – nested evaluation:

Loop	Data	Purpose
Outer	$\mathcal{D}_{\text{test}}$ (held out before any search)	Unbiased performance estimate
Inner	$\mathcal{D}_{\text{val}} \subset \mathcal{D} \setminus \mathcal{D}_{\text{test}}$	Drives AutoML search

- Nested CV estimates the performance of the *AutoML procedure*, not a configuration
- After nested evaluation, refit on all of $\mathcal{D}$ for deployment
- Never let $\mathcal{D}_{\text{test}}$ influence any search or preprocessing decision

5. Visualizing AutoML Runs

Why anytime curves? A single final score conflates systems with very different search dynamics.

Key properties of anytime performance curves:

- Show the incumbent risk $\widehat{\mathcal{R}_{\text{gen}}}(\boldsymbol{\lambda}_t^*)$ as a function of time t
- Curves are right-continuous step functions – linear interpolation is misleading
- Enable budget-independent conclusions (anytime superiority) vs. crossover behaviour

Aggregating multiple runs:

- Compute the pointwise mean on a common time grid from n independent runs
- Use **standard error (SE)** bands for comparing methods: $\mathrm{SE}(t) = \sigma(t)/\sqrt{n}$
- Use standard deviation (SD) only to characterize run-to-run variability
- Non-overlapping SE bands are indicative but not conclusive

6. Benchmarking AutoML

Benchmarking asks: how does a method compare across many problems?

Desiderata for a fair benchmark:

- Controlled resources (time, memory, hardware) for all systems
- Nested evaluation; diverse task suite; simple baselines always included
- Reproducible: fixed splits, public data, containerised environments
- Open: code, splits, and raw results publicly available

Key benchmarks: OpenML-CC18, AutoML Benchmark (Gijsbers et al. 2024), HPO-B, YAHPO Gym, PD1, CARP-S.

Aggregation: Normalise scores per dataset (rank-based or relative scaling), then apply Friedman + post-hoc tests and visualise with critical difference diagrams.

Threats to validity: cherry-picking datasets, hardware heterogeneity, dataset leakage, metric choice sensitivity, single-run comparisons.

Chapter Summary

Key takeaways from Chapter 2:

1. Evaluation is our proxy for generalization: **weak eval makes progress an illusion**
2. Any finite-data estimator of $\mathcal{R}^{(n)}_{\text{gen}}(\boldsymbol{\lambda})$ has bias and variance – choose a resampling scheme according to data size and computational budget
3. Statistical tests are required to distinguish real effects from random variation; **always correct for multiple comparisons**
4. AutoML amplifies evaluation bias through data-dependent selection – **nested evaluation** is mandatory to obtain unbiased performance estimates
5. AutoML systems are anytime algorithms – **anytime curves with SE bands** capture behaviour that a single final score cannot
6. Credible benchmarks require diverse task suites, controlled resources, nested evaluation, multiple runs, and open artefacts; **cherry-picking datasets is a serious threat to validity**